



ECI HRM Performance Appraisal System

Internal Server Deployment Guide



Version 1.0 — Production Release

June 2025

CONTENTS

Table of Contents

01	Server Preparation	
02	Software & Tools Installation	
03	Code Deployment	
04	Process Management	
05	Reverse Proxy & Access	
06	Backup & Recovery	
07	Testing & Verification Checklist	
08	Production Launch & Maintenance	

CHAPTER 1

Server Preparation

Before deploying the ECI HRM Performance Appraisal System, the target server must meet specific hardware and software requirements. This chapter outlines the minimum specifications, operating system options, network configuration steps, and firewall rules necessary to prepare a production-ready environment. Proper server preparation is the foundation of a reliable and performant deployment.

1.1 Hardware Requirements

The server must meet the following minimum hardware specifications to ensure smooth operation under typical office loads of 20 to 100 concurrent users. While the application can run on lower specifications for evaluation purposes, production deployments should not compromise on these baseline resources.

COMPONENT	MINIMUM	RECOMMENDED
CPU	4 cores (x86_64)	8 cores (x86_64)
RAM	8 GB DDR4	16 GB DDR4
Storage	100 GB SSD	256 GB NVMe SSD
Network	Gigabit Ethernet	Gigabit Ethernet
Backup	External drive / NAS	Dedicated NAS with RAID

The SSD requirement is particularly important. PostgreSQL performs significantly better on SSD storage compared to traditional HDDs, especially for concurrent read/write operations during appraisal cycles when multiple users are simultaneously accessing the system. For organizations expecting heavy usage or planning to store uploaded documents (evidence files, review attachments), allocate at least 256 GB of storage.

1.2 Operating System Options

The ECI HRM system supports deployment on both Linux and Windows platforms. The following operating systems have been tested and are fully supported for production use. Select the platform that best matches your organization's existing infrastructure and the skill set of your IT team.

PLATFORM	OS	STATUS
Linux	Ubuntu 22.04 LTS / 24.04 LTS	Recommended
Linux	Debian 12 (Bookworm)	Supported
Windows	Windows Server 2019 / 2022	Supported
Windows	Windows 10 Pro / 11 Pro	Supported

Ubuntu Server 22.04 LTS or 24.04 LTS is the recommended platform. It offers long-term support, excellent PostgreSQL performance, native support for PM2 process management, and Nginx reverse proxy capabilities. Most of the commands in this guide are written with Ubuntu in mind, though equivalent Windows instructions are provided for every critical step.

Windows deployments use IIS (Internet Information Services) with the Application Request Routing (ARR) module as the reverse proxy, and NSSM (Non-Sucking Service Manager) for process management. While fully functional, the Windows path requires additional configuration steps and is better suited for organizations that lack Linux administration expertise.

1.3 Static IP Configuration

The server must be assigned a static local IP address so that office workstations can reliably connect to it. A DHCP reservation is also acceptable, but a manually configured static IP is preferred for simplicity and predictability. The IP address should be outside the DHCP pool range to avoid conflicts.

Linux (Ubuntu) — Netplan Configuration:

On Ubuntu, network configuration is managed through Netplan. Edit the appropriate YAML file in `/etc/netplan/` and set a static address. The following example configures the server with IP address `192.168.1.50` :

```
# /etc/netplan/01-static-ip.yaml
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: no
      addresses:
        - 192.168.1.50/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Apply the configuration with `sudo netplan apply` and verify connectivity with `ip addr show eth0` . Ensure you can ping the gateway and external addresses before proceeding.

Windows — Static IP via GUI:

On Windows, open **Control Panel > Network and Sharing Center > Change adapter settings**. Right-click the active network adapter, select **Properties**, then **Internet Protocol Version 4 (TCP/IPv4)**. Enter the static IP, subnet mask (typically `255.255.255.0`), default gateway, and DNS servers. Click OK to apply and verify with `ipconfig /all` in a Command Prompt.

1.4 Network Topology

The recommended network topology follows a simple and reliable pattern. The ECI HRM server connects directly to the office network switch using a Gigabit Ethernet cable. All office PCs connect to the same switch, enabling local network access without requiring internet connectivity. This air-gapped approach ensures maximum data security for sensitive employee performance data.

The network flow is: **Office PCs ↔ Network Switch ↔ ECI HRM Server**. Each workstation accesses the system through a browser at `http://hrm.eci.local` . DNS resolution is handled either by the office DNS server (preferred) or by adding a hosts file entry on each workstation. For organizations with a dedicated DNS server, create an A record mapping `hrm.eci.local` to the server's static IP. For smaller offices without DNS infrastructure, add an entry to the Windows hosts file at `C:\Windows\System32\drivers\etc\hosts` on each PC.

1.5 Firewall Configuration

The server firewall must allow traffic on the following ports for the system to function correctly. Restrict access to the office network subnet where possible to enhance security. Do not expose these ports to the public internet under any circumstances.

PORT	SERVICE	PURPOSE	ACCESS
3000	Node.js (Next.js)	Application server	Localhost only
5432	PostgreSQL	Database server	Localhost only
80	Nginx / IIS	Reverse proxy (HTTP)	Office subnet
443	Nginx (optional)	Reverse proxy (HTTPS)	Office subnet

Linux (UFW):

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.1.0/24 to any port 80
sudo ufw allow from 192.168.1.0/24 to any port 443
sudo ufw allow 3000/tcp      # localhost only (bound to 127.0.0.1)
sudo ufw allow 5432/tcp      # localhost only (bound to 127.0.0.1)
sudo ufw enable
sudo ufw status verbose
```

Windows Firewall:

```
# Allow HTTP from office subnet
New-NetFirewallRule -DisplayName "ECI-HRM HTTP" `
  -Direction Inbound -Protocol TCP -LocalPort 80 `
  -RemoteAddress 192.168.1.0/24 -Action Allow

# Allow HTTPS (optional)
New-NetFirewallRule -DisplayName "ECI-HRM HTTPS" `
  -Direction Inbound -Protocol TCP -LocalPort 443 `
  -RemoteAddress 192.168.1.0/24 -Action Allow
```

IMPORTANT

Ports 3000 (Node.js) and 5432 (PostgreSQL) should **only** listen on `127.0.0.1` (localhost). They must never be exposed to the network directly. All external traffic must flow through the reverse proxy (Nginx or IIS) on port 80.

CHAPTER 2

Software & Tools Installation

This chapter covers the installation of all required software components on the server. Each tool serves a specific role in the deployment stack: Node.js runs the application, PostgreSQL stores the data, Git retrieves the source code, PM2 (or NSSM) manages the process, and Nginx (or IIS) acts as the reverse proxy. Follow the instructions for your chosen platform carefully, verifying each step before proceeding to the next.

2.1 Node.js 22.x LTS Installation

Node.js is the JavaScript runtime that powers the Next.js application. The Long-Term Support (LTS) version 22.x provides stability and security updates for an extended period. Always install the LTS version rather than the latest feature release to ensure production reliability.

Linux (Ubuntu) — via NodeSource:

```
# Update system packages
sudo apt update && sudo apt upgrade -y

# Install Node.js 22.x LTS via NodeSource
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs

# Verify installation
node --version      # Expected: v22.x.x
npm --version       # Expected: 10.x.x
```

Windows — via MSI Installer:

Download the Node.js 22.x LTS MSI installer from <https://nodejs.org>. Run the installer and accept all default options. The MSI installer automatically adds Node.js and npm to the system PATH. Verify the installation by opening a new Command Prompt and running `node --version` and `npm --version`. If the commands are not recognized, restart the terminal or log out and back in to refresh the PATH environment variable.

2.2 PostgreSQL 16 Installation and Configuration

PostgreSQL is the production database for the ECI HRM system. While the development environment uses SQLite for simplicity, PostgreSQL provides the concurrency, reliability, and data integrity required for multi-user production deployments. PostgreSQL 16 is the recommended version, though PostgreSQL 15 is also fully compatible.

Linux (Ubuntu):

```
# Add PostgreSQL APT repository
sudo apt install -y wget gnupg2 lsb-release
sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt \
$(lsb_release -cs)-pgdg main" > /etc/apt/sources.list.d/pgdg.list'
wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc \
| sudo apt-key add -

# Install PostgreSQL 16
sudo apt update
sudo apt install -y postgresql-16

# Start and enable the service
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

Windows:

Download the PostgreSQL 16 installer from <https://www.postgresql.org/download/windows/>. Run the installer and accept the default installation directory. During setup, set the superuser (postgres)

password to a strong value and record it securely. Leave the default port (5432) unchanged. Ensure the "Stack Builder" option is unchecked unless you need additional tools.

Create Database and User:

```
# Connect to PostgreSQL as superuser
sudo -u postgres psql

# Inside the PostgreSQL shell:
CREATE DATABASE eci_hrm;
CREATE USER hrm_user WITH PASSWORD 'YOUR_STRONG_PASSWORD_HERE';
GRANT ALL PRIVILEGES ON DATABASE eci_hrm TO hrm_user;

# Connect to the new database and grant schema permissions
\c eci_hrm
GRANT ALL ON SCHEMA public TO hrm_user;

# Exit
\q
```

SECURITY NOTE

Replace `YOUR_STRONG_PASSWORD_HERE` with a strong password (minimum 16 characters, mixed case, numbers, and special characters). This same password will be used in the `.env` file and `ecosystem.config.json`. Never use default or weak passwords in production.

2.3 Git Installation

Git is required to clone the application repository from the code server. On Ubuntu, Git is typically pre-installed. If not, install it with `sudo apt install -y git`. On Windows, download the installer from <https://git-scm.com/download/win> and use the default options. Verify with `git --version` after installation.

2.4 PM2 Installation (Linux) / NSSM (Windows)

A process manager ensures the Node.js application runs continuously, automatically restarts after crashes or server reboots, and provides log management capabilities.

Linux — PM2:

```
# Install PM2 globally
sudo npm install -g pm2

# Verify
pm2 --version
```

PM2 is a production-grade process manager for Node.js. It supports auto-restart on crash, log rotation, monitoring, and startup script generation. It is the recommended choice for Linux deployments and will be configured in Chapter 4.

Windows — NSSM:

Download NSSM from <https://nssm.cc/download> . Extract the ZIP archive and copy `nssm.exe` to a location in your system PATH (e.g., `C:\Windows\System32` or a dedicated tools directory added to PATH). NSSM wraps any command-line executable as a Windows Service, providing automatic startup and crash recovery equivalent to PM2 on Linux.

2.5 Nginx Installation (Linux) / IIS with ARR (Windows)

The reverse proxy sits in front of the Node.js application, handling HTTP requests from office workstations and forwarding them to the application on port 3000. It also provides security headers, gzip compression, and static file caching.

Linux — Nginx:

```
sudo apt install -y nginx
sudo systemctl start nginx
sudo systemctl enable nginx
sudo systemctl status nginx
```

Windows — IIS with ARR Module:

Open **Server Manager > Manage > Add Roles and Features**. Select "Web Server (IIS)" role. Under Role Services, ensure "Common HTTP Features", "Security", and "Performance" are all checked. After IIS is installed, download and install the **Application Request Routing (ARR)** module from the official Microsoft IIS website. Open IIS Manager, select the server node, double-click "Application Request Routing Cache", and choose "Server Proxy Settings" from the right-hand actions panel. Enable proxy functionality by checking "Enable proxy".

2.6 Application Directory Structure

Create the application directory before cloning the repository. The directory should have appropriate ownership and permissions.

Linux:

```
# Create application directory
sudo mkdir -p /opt/eci-hrm
sudo chown $USER:$USER /opt/eci-hrm

# Create log directory
sudo mkdir -p /var/log/eci-hrm
sudo chown $USER:$USER /var/log/eci-hrm
```

Windows:

```
# Create application directory (run in Command Prompt as Administrator)
mkdir C:\ECI-HRM
mkdir C:\ECI-HRM\logs
```


Code Deployment

This chapter covers the complete code deployment workflow: cloning the repository, installing dependencies, configuring the environment, switching the database provider from SQLite to PostgreSQL, building the production bundle, seeding the initial data, and verifying the build output. Each step must be completed in order, and every command should be verified before moving forward. Errors at this stage often stem from missing dependencies, incorrect environment variables, or database connection issues.

3.1 Clone the Repository

```
cd /opt/eci-hrm
git clone https://github.com/ECI-HRM/HRM.git .
# Note the "." at the end clones directly into the current directory
```

If the repository is private, ensure the deploying user has SSH key or personal access token authentication configured. For organizations using an internal Git server (GitLab, Gitea), replace the URL accordingly. Verify the clone by checking that the `package.json`, `prisma/`, and `src/` directories exist.

3.2 Install Dependencies

```
cd /opt/eci-hrm
npm install
```

This command reads `package.json` and installs all required Node.js modules into the `node_modules/` directory. On a fresh server with a good internet connection, this typically takes two to five minutes. If you encounter permission errors on Linux, ensure the `/opt/eci-hrm` directory is owned by the deploying user. If the install fails due to network issues, check proxy settings or try using an npm registry mirror.

3.3 Configure Environment Variables

Copy the example environment file and edit it with your production-specific values. The `.env` file contains all configuration that the application needs at runtime, including the database connection string, admin credentials, and the application URL.

```
# Copy the example file
cp .env.example .env

# Edit with your preferred editor
nano .env
```

Configure the following values in the `.env` file. Every value must be set correctly; do not leave any placeholder values unchanged.

```
# Database connection (PostgreSQL)
DATABASE_URL="postgresql://hrm_user:YOUR_STRONG_PASSWORD@localhost:5432/eci_hrm?
schema=public"

# Admin account (created during seed)
ADMIN_EMAIL=imunir@eci.com.pk
ADMIN_NAME=Muhammad Unir
ADMIN_PASSWORD=ECI@dm1n#2025!Secure
ADMIN_EMPLOYEE_ID=ECI-001

# Application URL (must match what users type in their browser)
NEXT_PUBLIC_APP_URL=http://hrm.eci.local

# Environment mode
NODE_ENV=production
```

CRITICAL

The default admin password `ECI@dm1n#2025!Secure` must be changed immediately after the first login. Do not skip this step. Also, ensure the `DATABASE_URL` password matches exactly what you set when creating the PostgreSQL user in Chapter 2.

3.4 Switch Prisma Schema to PostgreSQL

The development version of the application uses SQLite for convenience. For production, the database provider must be changed to PostgreSQL. Open the `prisma/schema.prisma` file and modify the `datasource` block as follows:

```
# prisma/schema.prisma - BEFORE (development)
datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

# prisma/schema.prisma - AFTER (production)
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

Only the `provider` line needs to change from `"sqlite"` to `"postgresql"`. The `url` line remains unchanged because it references the `DATABASE_URL` environment variable, which now points to PostgreSQL. Save the file after making this change.

3.5 Generate Prisma Client and Push Schema

```
# Generate the Prisma client for PostgreSQL
npx prisma generate

# Push the schema to the database (creates all tables)
npx prisma db push
```

The `prisma generate` command creates the Prisma Client library optimized for PostgreSQL. The `prisma db push` command reads the schema file and creates (or updates) all required tables, columns, indexes, and relations in the PostgreSQL database. If the database is empty, this creates the entire schema from scratch. If tables already exist, it applies any new migrations or changes incrementally without data loss.

Verify the schema was applied successfully by checking the output for "Your database is now in sync with your Prisma schema" or a similar confirmation message. If you see errors about missing extensions or permissions, revisit the PostgreSQL user permissions from Chapter 2.

3.6 Build the Production Bundle

```
# Build the Next.js production bundle
npm run build
```

This command compiles the entire application, optimizes assets, and generates the production-ready output in the `.next/` directory. The build process typically takes three to eight minutes depending on the server's CPU and RAM. The output should indicate "Build completed successfully" without any errors or warnings about missing modules. The `next.config.ts` file is configured to output a standalone server, which means the production build is self-contained and does not require the full `node_modules` directory to run.

3.7 Copy Static Files

```
# Copy static assets into the standalone build
cp -r .next/static .next/standalone/.next/
```

The standalone build produced by Next.js requires the static assets (images, fonts, CSS) to be manually copied into the standalone directory. This is a known requirement of Next.js standalone output mode. Without this step, the application will serve but all static assets (including the ECI logo and application styling) will return 404 errors. On Windows, the equivalent command is `xcopy .next\static .next\standalone\.next\static /E /I`.

3.8 Production Seed (Admin Account and Master Data)

```
# Seed ONLY production data (admin account + master data, NO demo users)
curl -X POST "http://localhost:3000/api/seed?mode=production"
```

The production seed creates the essential data required for the system to function, and critically, it does **not** create any demo or test user accounts. The following items are created by the production seed:

- **1 Admin Account:** Email: `imunir@eci.com.pk`, Name: Muhammad Unir, Employee ID: ECI-001, Password: `ECI@dm1n#2025!Secure`
- **3 Rating Scales:** Standard 5-point rating scale used across all appraisal forms for consistent evaluation
- **22 Appraisal Categories:** Pre-configured evaluation criteria covering goals, competencies, and behavioral indicators

The seed endpoint must be called while the application is running (either in development mode via `npm run dev` or in production mode via PM2/NSSM). If you have already started the process manager, use the running server URL. If not, start the server temporarily with `node .next/standalone/server.js` in a separate terminal, run the seed command, and then stop the temporary server with Ctrl+C.

NOTE

After seeding, log in with the admin account and **immediately change the default password**. The admin can then create departments, designations, employees, and configure the first appraisal cycle from within the application's admin dashboard.

CHAPTER 4

Process Management

Process management ensures the ECI HRM application runs continuously as a background service, automatically recovers from crashes, and starts automatically when the server reboots. Without a process manager, a terminal session that is accidentally closed would terminate the application, causing downtime. This chapter covers configuration for both PM2 on Linux and NSSM on Windows, including auto-restart on boot, log management, and monitoring commands.

4.1 Linux Process Management with PM2

The project includes a pre-configured `ecosystem.config.json` file that defines how PM2 should run the application. This file specifies the script path, environment variables, log file locations, restart policies, and memory limits. Before using it, update the `DATABASE_URL` in the ecosystem file to match your production password.

Start the Application:

```
cd /opt/eci-hrm
pm2 start ecosystem.config.json
```

PM2 reads the `ecosystem.config.json` file and starts the application with all configured settings. Verify it is running with `pm2 status`, which should show the `eci-hrm` process in "online" state. If the status shows "errored" or "stopped", check the logs with `pm2 logs eci-hrm --lines 50` for error

details. Common startup issues include incorrect DATABASE_URL, missing static files, or port 3000 already in use.

Configure Auto-Restart on Server Boot:

```
# Save the current process list
pm2 save

# Generate and install the startup script
pm2 startup

# The command above outputs a sudo command. Copy and run it:
# Example output: sudo env PATH=$PATH:/usr/bin /usr/lib/node_modules/pm2/bin/pm2
startup...
# Run that exact sudo command as instructed.
```

The `pm2 startup` command generates a system service script specific to your init system (systemd on Ubuntu 22.04/24.04). Running the generated sudo command registers PM2 as a system service, so it starts automatically when the server boots and respawns the application. The `pm2 save` command persists the current process list to disk so PM2 knows which processes to restore after a reboot.

Monitoring and Logs:

```
# View real-time logs (Ctrl+C to exit)
pm2 logs eci-hrm

# Open the monitoring dashboard
pm2 monit

# Check process status
pm2 status

# View detailed process info
pm2 show eci-hrm

# Restart the application (after config changes)
pm2 restart eci-hrm

# Stop the application
pm2 stop eci-hrm

# Delete the process from PM2 (use with caution)
pm2 delete eci-hrm
```

The `pm2 monit` command opens a real-time terminal dashboard showing CPU usage, memory consumption, and log output. It is invaluable for diagnosing performance issues. The `pm2 logs` command streams both stdout and stderr logs to the terminal. Log files are also written to `/var/log/eci-hrm/out.log` and `/var/log/eci-hrm/error.log` as configured in `ecosystem.config.json`.

4.2 Windows Process Management with NSSM

On Windows, NSSM (Non-Sucking Service Manager) wraps the Node.js process as a Windows Service. This provides the same benefits as PM2: automatic startup on boot, crash recovery, and integration with Windows Service management tools. The following commands should be run in an elevated Command Prompt (Run as Administrator).

Install and Configure the Service:

```
# Install the service
nssm install ECI-HRM "C:\Program Files\nodejs\node.exe"

# Set the application parameters
nssm set ECI-HRM AppParameters "C:\ECI-HRM\next\standalone\server.js"

# Set the working directory
nssm set ECI-HRM AppDirectory "C:\ECI-HRM"

# Set environment variables
nssm set ECI-HRM AppEnvironmentExtra NODE_ENV=production
nssm set ECI-HRM AppEnvironmentExtra PORT=3000
nssm set ECI-HRM AppEnvironmentExtra HOSTNAME=0.0.0.0
nssm set ECI-HRM AppEnvironmentExtra
DATABASE_URL=postgresql://hrm_user:YOUR_PASSWORD@localhost:5432/eci_hrm?schema=public
nssm set ECI-HRM AppEnvironmentExtra ADMIN_EMAIL=imunir@eci.com.pk
nssm set ECI-HRM AppEnvironmentExtra ADMIN_NAME=Muhammad Unir
nssm set ECI-HRM AppEnvironmentExtra ADMIN_PASSWORD=ECI@dm1n#2025!Secure
nssm set ECI-HRM AppEnvironmentExtra ADMIN_EMPLOYEE_ID=ECI-001
nssm set ECI-HRM AppEnvironmentExtra NEXT_PUBLIC_APP_URL=http://hrm.eci.local

# Configure auto-start and restart
nssm set ECI-HRM Start SERVICE_AUTO_START
nssm set ECI-HRM AppStdout "C:\ECI-HRM\logs\out.log"
nssm set ECI-HRM AppStderr "C:\ECI-HRM\logs\error.log"

# Create logs directory
mkdir C:\ECI-HRM\logs

# Start the service
nssm start ECI-HRM
```

After starting the service, verify it is running with `nssm status ECI-HRM` (should return "SERVICE_RUNNING") or by checking the Windows Services console (`services.msc`). You can also open `http://localhost:3000` in a browser on the server to confirm the application is responding. To view logs, check the files in `C:\ECI-HRM\logs\` . To restart the service after configuration changes, use `nssm restart ECI-HRM` .

NSSM also provides a GUI configuration tool. Run `nssm install ECI-HRM` without command-line parameters to open the GUI, where you can set all fields through a visual interface. This can be helpful for administrators who prefer GUI-based configuration.

Reverse Proxy & Access

The reverse proxy is the entry point for all user traffic. It listens on port 80 (HTTP) or port 443 (HTTPS) and forwards requests to the Node.js application running on port 3000. This architecture provides several advantages: the application does not need to run as root to bind to port 80, the proxy handles SSL termination for HTTPS, and security headers and compression are applied uniformly. This chapter covers Nginx configuration for Linux and IIS configuration for Windows, along with DNS setup for the `hrm.eci.local` hostname.

5.1 Linux — Nginx Reverse Proxy Configuration

The project includes a pre-configured Nginx configuration file at `deploy/nginx-eci-hrm.conf`. This file is production-ready and includes security headers, gzip compression, WebSocket support, static file caching, and proper proxy headers. Follow these steps to deploy it:

```
# Copy the configuration file to Nginx sites-available
sudo cp /opt/eci-hrm/deploy/nginx-eci-hrm.conf /etc/nginx/sites-available/eci-hrm

# Create a symbolic link to enable the site
sudo ln -s /etc/nginx/sites-available/eci-hrm /etc/nginx/sites-enabled/

# Remove the default Nginx site (optional but recommended)
sudo rm -f /etc/nginx/sites-enabled/default

# Test the Nginx configuration for syntax errors
sudo nginx -t

# If the test passes, reload Nginx
sudo systemctl reload nginx
```

The `nginx -t` command validates the configuration syntax. If it reports errors, review the configuration file path and content. The most common issue is a missing semicolon or an incorrect `server_name` directive. After a successful reload, the application should be accessible at `http://hrm.eci.local` from any workstation on the network (once DNS is configured).

5.2 DNS Configuration

For office workstations to access the system via `http://hrm.eci.local`, the hostname must resolve to the server's IP address. There are two approaches depending on your network infrastructure:

Option A — DNS Server (Recommended): If your office has a DNS server (Windows Server DNS, dnsmasq, or a router with DNS capabilities), create an A record mapping `hrm.eci.local` to the server's static IP (e.g., `192.168.1.50`). This is the preferred method because it works for all workstations without any per-machine configuration.

Option B — Hosts File: If no DNS server is available, add an entry to the hosts file on each workstation. On Windows, edit `C:\Windows\System32\drivers\etc\hosts` (open as Administrator in Notepad). On Linux/macOS, edit `/etc/hosts`. Add the following line:

```
192.168.1.50    hrm.eci.local
```

Replace `192.168.1.50` with your server's actual static IP. After saving the hosts file, flush the DNS cache with `ipconfig /flushdns` (Windows) or `sudo systemd-resolve --flush-caches` (Ubuntu).

5.3 Optional HTTPS with Self-Signed Certificate

For internal deployments, HTTPS is optional but recommended if the organization handles sensitive performance data. A self-signed certificate encrypts traffic between workstations and the server, preventing network sniffing. Note that self-signed certificates will trigger browser warnings; users must accept the certificate once to proceed.

```
# Create SSL directory
sudo mkdir -p /etc/nginx/ssl

# Generate a self-signed certificate (valid for 365 days)
sudo openssl req -x509 -nodes -days 365 \
  -newkey rsa:2048 \
  -keyout /etc/nginx/ssl/eci-hrm.key \
  -out /etc/nginx/ssl/eci-hrm.crt \
  -subj "/C=PK/ST=Punjab/L=Lahore/O=ECI Pvt Ltd/CN=hrm.eci.local"

# Uncomment the HTTPS server block in the Nginx config
sudo nano /etc/nginx/sites-available/eci-hrm

# Test and reload
sudo nginx -t && sudo systemctl reload nginx
```

After enabling HTTPS, update `NEXT_PUBLIC_APP_URL` in the `.env` file and `ecosystem.config.json` to `https://hrm.eci.local`, then restart the application with `pm2 restart eci-hrm`. Also configure Nginx to redirect all HTTP traffic to HTTPS by adding a `return 301 https://$host$request_uri;` directive in the HTTP server block.

5.4 Windows — IIS Reverse Proxy Configuration

On Windows Server, IIS serves as the reverse proxy using the Application Request Routing (ARR) module. Follow these steps to configure IIS for the ECI HRM system:

1. Open **IIS Manager** (`inetmgr`)
2. Right-click **Sites** > **Add Website**
3. Site name: `ECI-HRM`
4. Physical path: `C:\inetpub\wwwroot\hrm.eci.local` (create this empty directory first)
5. Host name: `hrm.eci.local`

6. Click OK to create the site

```
# Create the site root directory
mkdir C:\inetpub\wwwroot\hrm.eci.local

# Copy the web.config file (ARR reverse proxy rules)
copy C:\ECI-HRM\deploy\iis-web.config C:\inetpub\wwwroot\hrm.eci.local\web.config
```

The `deploy/iis-web.config` file contains URL rewrite rules that proxy all incoming requests to `http://localhost:3000`. It also includes security headers (X-Frame-Options, X-Content-Type-Options, X-XSS-Protection) and a 50MB request size limit for file uploads. After copying the file, open a browser and navigate to `http://hrm.eci.local` to verify the proxy is working correctly. If you see a 502 or 503 error, confirm the Node.js service (NSSM) is running and listening on port 3000.

CHAPTER 6

Backup & Recovery

Regular backups are essential for any production system. The ECI HRM system stores critical employee data, performance appraisals, and organizational structure information that would be extremely difficult to reconstruct manually. This chapter establishes a two-tier backup strategy: daily database backups for point-in-time recovery, and weekly full backups that include configuration files and uploaded documents. Both Linux and Windows backup procedures are covered, along with restoration steps.

6.1 Daily Database Backup

A daily database backup captures the complete state of the PostgreSQL database. The backup script uses `pg_dump` with gzip compression to minimize storage usage, and automatically rotates old backups beyond the retention period. The project includes a ready-to-use backup script at `deploy/backup-db.sh` for Linux and `deploy/backup-db.bat` for Windows.

Linux — Schedule with Cron:

```
# Make the script executable
chmod +x /opt/eci-hrm/deploy/backup-db.sh

# Edit crontab for the application user
crontab -e

# Add this line to run daily at 2:00 AM
0 2 * * * /opt/eci-hrm/deploy/backup-db.sh >> /var/log/eci-hrm/backup.log 2>&1
```

Windows — Schedule with Task Scheduler:

```
# Create a scheduled task (run in elevated Command Prompt)
schtasks /create /tn "ECI-HRM Daily DB Backup" /tr "C:\ECI-HRM\deploy\backup-db.bat" \
/sc daily /st 02:00 /ru SYSTEM /rl HIGHEST
```

The daily backup script extracts the database password from the `.env` file, runs `pg_dump` with the `--clean --if-exists --no-owner --no-privileges` flags, compresses the output with gzip, and saves it to `/opt/eci-hrm/backups/db/` with a timestamp filename. Files older than 30 days are automatically deleted. Verify the first backup by checking that a `.sql.gz` file exists in the backup directory and is non-empty.

6.2 Weekly Full Backup

The weekly full backup captures not only the database but also the application configuration files (`.env`, `prisma/` schema), public assets (`public/` directory including the ECI logo), and any uploaded documents. This provides a complete snapshot that can restore the entire application to a working state on a fresh server. The backup script is located at `deploy/backup-full.sh`.

```
# Schedule weekly full backup (every Sunday at 3:00 AM)
# Add to crontab:
0 3 * * 0 /opt/eci-hrm/deploy/backup-full.sh >> /var/log/eci-hrm/backup.log 2>&1
```

The full backup creates a timestamped directory containing three compressed archives: `database.sql.gz` (the database dump), `app-files.tar.gz` (configuration and assets), and `uploads.tar.gz` (uploaded documents, if any). The `node_modules/`, `.next/cache/`, and `backups/` directories are excluded to save space. Weekly backups are retained for 12 weeks (approximately 3 months), after which the oldest directories are automatically removed.

6.3 Backup Storage

Backups should never be stored on the same physical drive as the application. In the event of a drive failure, both the application and its backups would be lost. The recommended approach is to copy backup files to a Network Attached Storage (NAS) device, an external USB hard drive, or a second internal drive. For maximum safety, implement the 3-2-1 backup rule: keep three copies of data, on two different media types, with one copy off-site.

```
# Example: Sync daily backups to a NAS mount
# Add to crontab (run after the daily backup):
30 2 * * * rsync -av --delete /opt/eci-hrm/backups/db/ /mnt/nas/eci-hrm-db/
```

6.4 Restore Procedure

In the event of data loss or corruption, the database can be restored from a backup. The restore process drops and recreates all tables, then imports the data from the backup dump. The following commands assume a Linux environment with an existing PostgreSQL user and database.

```
# Restore from a daily database backup
gunzip < /opt/eci-hrm/backups/db/eci_hrm_2025-06-15_020001.sql.gz \
| psql -U hrm_user -d eci_hrm

# If the database itself was dropped, recreate it first:
sudo -u postgres psql
DROP DATABASE eci_hrm;
CREATE DATABASE eci_hrm;
GRANT ALL PRIVILEGES ON DATABASE eci_hrm TO hrm_user;
\q

# Then restore:
gunzip < backup.sql.gz | psql -U hrm_user -d eci_hrm
```

After restoring, verify the data integrity by logging into the application and checking that employees, departments, and appraisal data are present. Run `npx prisma db push` if the backup is from an older version and the schema has changed. For full backups, extract the `app-files.tar.gz` archive to restore the `.env` and `prisma/` files, then rebuild the application with `npm run build`.

TEST YOUR BACKUPS

A backup that has never been tested for restoration is not a reliable backup. Schedule a quarterly restore test on a separate server or database instance to verify that backups are complete and uncorrupted. Document the restore procedure and time required so the team is prepared for an actual emergency.

CHAPTER 7

Testing & Verification Checklist

After deploying the ECI HRM system, a thorough testing process is essential to ensure every feature works correctly in the production environment. This chapter provides a comprehensive, sequential checklist that covers the complete appraisal workflow from login through final management approval and employee acknowledgment. Each test item should be verified and checked off before declaring the deployment complete. If any item fails, consult the troubleshooting section in Chapter 8 or review the relevant configuration step from earlier chapters.

The testing process follows the natural order of system usage: starting with authentication, then data setup, followed by the complete appraisal lifecycle, and finally reporting and role-based access verification. Tests should be performed from a workstation on the office network (not from the server itself) to accurately simulate real-world usage conditions.

Authentication and Dashboard

- 1 **Login/Logout:** Navigate to `http://hrm.eci.local` in a browser. Log in with `imunir@eci.com.pk` and the default password. Verify the login succeeds, the dashboard loads, and the logout function works correctly. Clear the browser cache between tests if needed.
- 2 **Dashboard Rendering:** Confirm the admin dashboard displays all stat cards (total employees, active cycles, pending reviews, completion rate), navigation sidebar, and header with user information. No console errors should appear in the browser developer tools.

Master Data Setup

- 3 **Create Departments:** Navigate to **Master Data > Departments**. Create at least three departments (e.g., Engineering, Finance, Human Resources). Verify they appear in the department list and can be edited or deactivated.
- 4 **Create Designations:** Navigate to **Master Data > Designations**. Create at least three designations (e.g., Software Engineer, Senior Accountant, HR Manager). Assign each designation to the appropriate department.
- 5 **Create Employees:** Navigate to **Master Data > Employees**. Create several test employees with correct department and designation assignments. For at least one employee, enable the **dual-role supervisor toggle** to allow them to act as both an employee and a supervisor.
- 6 **Set Line Manager Relationships:** Assign line managers (supervisors) to employees using the supervisor assignment feature. Verify the reporting structure is displayed correctly in the employee list view.

Appraisal Lifecycle

- 7 **Create and Activate Appraisal Cycle:** Navigate to **Appraisal Cycles**. Create a new cycle with a name, start date, and end date. Click **Activate** — this should auto-assign the cycle to all employees. Verify the cycle status changes to "Active" and assignments are created.
- 8 **Employee Self-Assessment:** Log in as a regular employee (not the admin). Navigate to the active appraisal cycle. Fill out the self-assessment form including goals, competencies, and any additional comments. Submit the self-assessment.
- 9 **Supervisor Review:** Log in as the assigned supervisor. Navigate to the pending review. Rate the employee on goals and competencies. Add supervisor comments. Submit the review.
- 10 **HR Review and Finalization:** Log in as the HR admin. Review the supervisor-submitted appraisal. Finalize the review with any additional HR comments. Confirm the status advances to "Finalized".
- 11 **Management Approval:** If the management role is configured, log in as a management user. Approve the finalized appraisal. Confirm the status changes to "Approved".
- 12 **HR Share with Employee:** As HR admin, share the approved appraisal with the employee. Log in as the employee and **acknowledge** receipt. Confirm the appraisal status shows "Acknowledged".

Reports, Notifications, and Role Verification

- 13 **View Reports and Export Data:** Navigate to the **Reports** section. Verify that appraisal summary reports, employee performance data, and cycle statistics are displayed correctly. Test the export functionality (CSV/Excel) to confirm data downloads properly.
- 14 **Check Notification Delivery:** Verify that in-app notifications are generated at each stage transition (cycle assigned, self-assessment due, review submitted, appraisal shared). Check the notification panel for each user role.
- 15 **Verify Sidebar Navigation per Role:** Log in as each role type (Admin, HR, Supervisor, Employee, Management) and confirm the sidebar shows only the navigation items appropriate for that role. Admin should see all sections; employees should see only their appraisals and profile.
- 16 **Test AI Analysis (if configured):** If an AI API key is configured in the environment variables, test the AI-powered performance analysis feature. Verify it generates meaningful insights and displays them correctly in the appraisal view.

- 17 Verify All Dashboards Load per Role:** Access the dashboard as Admin, Supervisor, Employee, and Management users. Confirm each role-specific dashboard displays the correct stat cards, charts, and quick-action buttons. No JavaScript errors should appear in the browser console.

TESTING TIP

Perform these tests from at least two different workstations to verify network accessibility. If any test fails, document the exact error message, browser console output, and server logs (`pm2 logs eci-hrm` or `C:\ECI-HRM\logs\error.log`) before troubleshooting.

CHAPTER 8

Production Launch & Maintenance

After all testing is complete and every checklist item has been verified, the system is ready for production launch. This chapter covers the critical first-day actions (changing the admin password, securing the seed endpoint), establishes a regular maintenance schedule, and provides troubleshooting guidance for the most common issues that may arise during operation. Following these procedures ensures the system remains secure, performant, and reliable throughout its production lifecycle.

8.1 First-Day Critical Actions

Before announcing the system to users, complete these mandatory security actions. Failure to perform these steps leaves the system vulnerable to unauthorized access.

Change the Admin Password: Log in with the default admin account (`imunir@eci.com.pk` / `ECI@dm1n#2025!Secure`) and immediately navigate to the profile or account settings to change the password. Choose a strong password that is not used for any other system. The default password is documented in this guide and must be considered compromised from the moment the guide is distributed.

Secure the Seed Endpoint: The `/api/seed` endpoint should be restricted or removed after the initial production seed is complete. Leaving this endpoint accessible allows anyone with network access to re-seed the database, potentially overwriting production data. Options include removing the route file entirely (`src/app/api/seed/route.ts`) and rebuilding, or adding authentication middleware to the route so only admin users can access it.

8.2 Regular Maintenance Schedule

A proactive maintenance schedule prevents issues before they affect users. The following tasks should be performed at their specified intervals by the designated IT administrator. Document all maintenance activities in a log for audit purposes and historical reference.

Weekly Tasks:

- **Check PM2 Logs:** Run `pm2 logs eci-hrm --lines 100` (Linux) or review `C:\ECI-HRM\logs\error.log` (Windows). Look for recurring errors, unhandled exceptions, or memory warnings.
- **Review Error Rates:** Monitor Nginx access and error logs at `/var/log/nginx/`. A spike in 5xx errors indicates application issues that need investigation.
- **Verify Backup Completion:** Confirm that daily backups are being created. Check the backup directory for files with today's timestamp and verify they are non-empty.
- **Check Disk Space:** Ensure the server has at least 20% free disk space. Database files, logs, and backups grow over time. Run `df -h` (Linux) or check Disk Management (Windows).

Monthly Tasks:

- **Update Dependencies:** Run `npm audit` to check for known security vulnerabilities in Node.js packages. Review the output and apply patches as recommended. Test thoroughly after any dependency updates.
- **Test Restore Procedure:** Perform a test restoration of the most recent backup on a separate database instance. This verifies both the backup integrity and the restore procedure. Record the time required for restoration.
- **Review Application Logs:** Scan the full month's error logs for patterns or recurring issues that may indicate underlying problems requiring code fixes or configuration adjustments.

Quarterly Tasks:

- **Review User Access:** Audit all user accounts. Remove or deactivate accounts for employees who have left the organization. Verify that role assignments (Admin, HR, Supervisor, Management) are still appropriate.
- **Database Maintenance:** Run `VACUUM ANALYZE` on the PostgreSQL database to reclaim space and update query planner statistics. This can be done via `sudo -u postgres psql -d eci_hrm -c "VACUUM ANALYZE;"`.
- **Security Review:** Verify firewall rules are still appropriate, check for any unauthorized access attempts in server logs, and review the Nginx/IIS configuration for any needed updates.
- **Update Node.js:** Check for Node.js LTS version updates. If a new LTS version is available, test the application on the new version in a staging environment before upgrading the production server.

8.3 Troubleshooting Common Issues

The following table covers the most common issues encountered during production operation, their likely causes, and step-by-step resolution procedures. Always check the application logs first when diagnosing any issue, as they provide the most specific error information.

ISSUE	LIKELY CAUSE	RESOLUTION
Server won't start	Port 3000 already in use, or .env DATABASE_URL is incorrect	Check port: <code>sudo lsof -i :3000</code> . Kill conflicting process or change PORT. Verify DATABASE_URL has correct password and hostname.
Blank page after login	Static files not copied, or build is outdated	Rebuild: <code>npm run build</code> . Re-copy static files: <code>cp -r .next/static .next/standalone/.next/</code> . Restart PM2/NSSM.
Database connection failed	PostgreSQL not running, or credentials mismatch	Check service: <code>sudo systemctl status postgresql</code> . Test connection: <code>psql -U hrm_user -d eci_hrm -c "SELECT 1;"</code>
Slow page loads	Large database, missing indexes, or low memory	Run <code>VACUUM ANALYZE</code> . Check indexes with <code>EXPLAIN ANALYZE</code> . Increase server RAM. Restart PM2: <code>pm2 restart eci-hrm</code> .
502 Bad Gateway	Node.js crashed or not running behind Nginx	Check PM2 status: <code>pm2 status</code> . If stopped, check logs: <code>pm2 logs eci-hrm --err --lines 50</code> . Restart if needed.
CSS/JS not loading	Static files missing from standalone build	Verify: <code>ls .next/standalone/.next/static/</code> . If empty, re-copy: <code>cp -r .next/static .next/standalone/.next/</code>

8.4 Updating the Application

When a new version of the ECI HRM application is released, follow this update procedure to minimize downtime and ensure a smooth transition. Always test the update on a staging environment before applying it to production.

```
# 1. Backup the current deployment and database
/opt/eci-hrm/deploy/backup-full.sh

# 2. Pull the latest code
cd /opt/eci-hrm
git pull origin main

# 3. Install updated dependencies
npm install

# 4. If the Prisma schema changed, update the database
npx prisma generate
npx prisma db push

# 5. Rebuild the production bundle
npm run build
cp -r .next/static .next/standalone/.next/

# 6. Restart the application
pm2 restart eci-hrm

# 7. Verify the application is running
pm2 logs eci-hrm --lines 20
curl -s -o /dev/null -w "%{http_code}" http://localhost:3000
```

If the update includes database schema changes, the `prisma db push` command will apply them. If it fails due to data incompatibility, consult the release notes for manual migration steps. In case of a failed update, restore from the backup taken in step 1 and report the issue to the development team.

8.5 Support and Escalation

For issues that cannot be resolved using the troubleshooting guide, contact the ECI HRM development team. When reporting an issue, include the following information: a description of the problem, steps to reproduce, the exact error message, browser console output (if applicable), server logs from the time of the error, and the current application version (available from `git log --oneline -1` in the application directory).

For urgent production outages that affect all users, prioritize service restoration over root cause analysis. Use the backup restore procedure if necessary to get the system back online, then investigate the root cause separately. Document all incidents and resolutions in a central log for future reference and continuous improvement of the deployment process.